

Đặc tả Thiết kế Kiểm thử (Test Design Specification)

Fintech Core Service Layer — Hệ thống lõi ngân hàng kiểu T24

Phiên bản	1.0
Ngày	2026-09-15
Tài liệu tham chiếu	SRS.md / SRS.pdf (đặc tả yêu cầu — cùng thư mục dự án)
Đối tượng sử dụng	QA kiểm thử API/nghiệp vụ (thủ công hoặc tự động hoá qua script)

1. Giới thiệu

1.1 Mục đích

Tài liệu đặc tả **thiết kế kiểm thử** (test design) cho toàn bộ 14 module chức năng của hệ thống, gồm: chiến lược kiểm thử, môi trường/dữ liệu cần chuẩn bị, và danh sách **146 test case** cụ thể (mã lỗi, dữ liệu vào, kết quả mong đợi, **câu lệnh SQL kiểm chứng trực tiếp trong DB**) — đủ chi tiết để QA thực thi qua REST client (curl/Postman/script) **và** xác nhận độc lập ở tầng dữ liệu, không chỉ tin vào response API.

Lưu ý quan trọng về schema DB: dự án Supabase này có 2 schema chứa bảng trùng tên (`public` và `t24`) — `t24` là dữ liệu có sẵn từ trước, **không liên quan** tới hệ thống trong tài liệu này. Toàn bộ câu lệnh SQL dưới đây đã ghi rõ tiền tố `public.` — chạy đúng theo đó để tránh nhầm sang schema `t24` .

Có 1 tài liệu phụ đi kèm: `ANSWER_KEY.md` (`ANSWER_KEY.pdf`) — cung cấp sẵn bộ dữ liệu đầu vào cụ thể + đáp án đã tính toán chính xác cho các test case có công thức (lãi suất, phân bổ gốc/lãi, quy đổi ngoại tệ, risk score...), để QA đối chiếu trực tiếp mà không cần tự tính tay.

1.2 Phạm vi kiểm thử

- Trong phạm vi:** kiểm thử API (tầng Controller→Service→Repository→DB) cho toàn bộ 14 module theo `SRS.md` mục 4; kiểm thử các quy tắc nghiệp vụ khó (tính lãi, hạch toán GL, kiểm soát kép, phân loại nợ xấu); smoke test giao diện Web Console.
- Ngoài phạm vi:** kiểm thử hiệu năng/tải (performance/load testing), kiểm thử bảo mật chuyên sâu (penetration testing), kiểm thử tự động hoá UI (không có framework FE để viết E2E UI test — Console chỉ là lớp mỏng gọi thẳng API nên kiểm thử API đã bao phủ phần lớn logic).

1.3 Mức độ ưu tiên (Priority)

MỨC	Ý NGHĨA
P1	Bắt buộc chạy mỗi lần release — nghiệp vụ lỗi hoặc rủi ro tiền/bảo mật cao
P2	Nên chạy mỗi lần release — quy tắc nghiệp vụ phụ, validation
P3	Chạy khi có thời gian / hồi quy định kỳ — case phụ, ít rủi ro

2. Chiến lược & phương pháp kiểm thử

KỸ THUẬT	ÁP DỤNG CHO
Equivalence Partitioning	Phân vùng amount hợp lệ/không hợp lệ, currency khớp/lệch, role hợp lệ/không hợp lệ
Boundary Value Analysis	Ngưỡng <code>FRAUD_AMOUNT_THRESHOLD</code> (đúng ngưỡng vs vượt 1 đồng), ngưỡng nợ xấu 90 ngày (90 vs 91 ngày), rút trước hạn (0 ngày vs 1 ngày)
State Transition Testing	Máy trạng thái <code>accounts</code> (active/dormant/frozen/closed), <code>cards</code> (active/blocked/expired), <code>loans</code> (active/closed/defaulted), <code>fraud_alerts</code> (open/reviewing/closed_fp/closed_confirmed), <code>term_deposits</code> (active/matured/withdrawn/closed), <code>auth_queue</code> (pending/authorized/rejected), <code>customers.kyc_status</code> (pending/verified/rejected)
Decision Table	Quy tắc RBAC Maker-Checker (role × hành động × là/không là chính người nộp lệnh)
Kịch bản end-to-end nhiều bước	Mở sổ tiết kiệm → trả lãi tháng → đáo hạn tái tục; tạo khoản vay → trả góp → đóng khoản vay; maker nộp lệnh → checker duyệt → kiểm tra tiền đã di chuyển
Kiểm tra bất biến hệ thống (invariant)	<code>GET /gl/trial-balance</code> phải luôn <code>balanced: true</code> — chạy lại sau mọi kịch bản khác như 1 bước hồi quy bắt buộc
Kiểm chứng song song ở tầng DB	Mỗi test case đều có kèm câu lệnh SQL (<code>psql</code> /Supabase SQL Editor) đọc thẳng bảng liên quan — phát hiện được trường hợp API trả response "trông đúng" nhưng dữ liệu ghi sai (hoặc ghi trùng do lỗi idempotency) mà chỉ nhìn response không thấy được. Quy tắc áp dụng cho cả test case âm (expect lỗi): nếu bước test gọi 1 endpoint có khả năng ghi dữ liệu (<code>POST</code> / <code>PATCH</code> / <code>PUT</code> / <code>DELETE</code>) và mong đợi bị từ chối (4xx), vẫn phải có câu SQL xác nhận không có bản ghi nào bị tạo/sửa/ngầm trước khi validation chặn lại — không được chỉ tin vào mã lỗi trả về, vì lỗi ứng dụng có thể khiến ghi dữ liệu xảy ra <i>trước</i> bước validate, hoặc thiếu rollback khi transaction lỗi giữa chừng. Chỉ những test case gọi <code>GET</code> (không có đường ghi) mới thực sự "không cần" kiểm DB — mọi trường hợp (không cần) khác trong tài liệu này đều phải ghi rõ lý do, không để mặc định.

3. Môi trường & dữ liệu kiểm thử

3.1 Chuẩn bị môi trường

1. Chạy đủ 7 migration theo thứ tự (`scripts/migrate.mjs` hoặc từng file `db/migrations/00X_*.sql`).
2. Khởi động server: `npm run dev` → `http://localhost:3000` .
3. Base URL API: `http://localhost:3000/api/v1` . Swagger UI (tham chiếu nhanh schema): `http://localhost:3000/docs` .

3.2 Dữ liệu có sẵn sau migration (không cần tạo lại)

BẢNG	DỮ LIỆU SEED
<code>account_types</code>	<code>id=1, type_name='Savings', interest_rate=2.50</code>
<code>transaction_categories</code>	<code>Deposit(income), Withdrawal(spending), Transfer(transfer), Interest(income)</code>
<code>gl_accounts</code>	7 tài khoản kế toán (mục 3.3 <code>SRS.md</code>)
<code>app_users</code>	5 user, mật khẩu <code>Demo@123</code> cho tất cả (bảng dưới)

USERNAME	ROLE	DÙNG ĐỂ TEST
<code>teller1</code>	maker	Nộp lệnh (không được tự duyệt)
<code>teller2</code>	maker	Nộp lệnh (dùng khi cần 2 maker khác nhau)
<code>supervisor1</code>	checker	Duyệt/từ chối lệnh của teller1/teller2
<code>supervisor2</code>	checker	Duyệt/từ chối (dùng khi cần 2 checker khác nhau)
<code>admin1</code>	both	Vừa nộp vừa duyệt (nhưng không tự duyệt lệnh của chính mình)

3.3 Dữ liệu cần tự tạo trước khi chạy bộ test (test data setup)

Thực hiện theo thứ tự, ghi lại ID trả về để dùng xuyên suốt các test case:

1. Tạo ≥2 `customers` (vd "Khách hàng A", "Khách hàng B").
2. Mở ≥2 `accounts` currency `VND` cho khách hàng A (1 tài khoản dùng để test rút tiền/đóng tài khoản, 1 tài khoản "sạch" dùng làm tài khoản đích cho transfer/FX).
3. Mở 1 `accounts` currency `USD` (dùng cho FR-FX) và nộp tiền USD vào đó qua `POST /transactions` (`txn_type=deposit` , `currency=USD` khớp với tài khoản).
4. Tạo 1 `exchange_rates` : `currency_from=USD, currency_to=VND, rate=25000, rate_date=<hôm nay>` .

5. Ghi nhớ: mọi ngày tháng dùng trong test rút trước hạn/đáo hạn/COB nên dùng **ngày quá khứ tính động** (vd "hôm nay trừ N ngày") thay vì hard-code, để bộ test không hết hạn theo thời gian thực.

4. Tiêu chí Entry/Exit

Entry (bắt đầu kiểm thử): migration chạy xong không lỗi; `GET /` và `GET /docs` trả 200; dữ liệu mục 3.3 đã tạo xong.

Exit (kết thúc/đạt release):

- 100% test case P1 pass.
- $\geq 95\%$ test case P2 pass (case fail còn lại phải có phiếu lỗi kèm mức độ nghiêm trọng).
- `GET /gl/trial-balance` trả `balanced: true` tại thời điểm kết thúc bộ test (bất biến kế toán không bị vi phạm bởi bất kỳ test nào).

5. Bảng tra cứu mã lỗi (Error Code Reference)

Tổng hợp **toàn bộ** mã lỗi nghiệp vụ trong hệ thống (trích trực tiếp từ `Errors.*` trong mã nguồn) — dùng để QA đối chiếu `error.code` trong response thay vì đoán.

HTTP	MÃ LỖI	KHI NÀO XẢY RA
400	<code>IDEMPOTENCY_KEY_REQUIRED</code>	Thiếu header <code>Idempotency-Key</code> ở endpoint bắt buộc
409	<code>IDEMPOTENCY_KEY_CONFLICT</code>	Cùng key, khác nội dung request
422	<code>VALIDATION_ERROR</code>	Lỗi zod schema, hoặc <code>Errors.validation(...)</code> (vd sai currency, sai amount)
404	<code>ROUTE_NOT_FOUND</code>	Endpoint không tồn tại
404	<code><RESOURCE>_NOT_FOUND</code> (vd <code>ACCOUNT_NOT_FOUND</code> , <code>CUSTOMER_NOT_FOUND</code> , <code>LOAN_NOT_FOUND</code> , <code>TERM_DEPOSIT_NOT_FOUND</code> , <code>TRANSACTION_NOT_FOUND</code> , <code>CARD_NOT_FOUND</code> , <code>APP_USER_NOT_FOUND</code> , <code>AUTH_QUEUE_NOT_FOUND</code> , <code>GL_ACCOUNT_NOT_FOUND</code> , <code>COB_RUN_NOT_FOUND</code> , <code>MERCHANT_NOT_FOUND</code> , <code>EXCHANGE_RATE_NOT_FOUND</code> ...)	Không tìm thấy bản ghi theo id
401	<code>UNAUTHORIZED</code>	Sai mật khẩu / thiếu-sai token JWT
403	<code>FORBIDDEN</code>	Đúng token nhưng sai vai trò (RBAC)
409	<code>CUSTOMER_EMAIL_TAKEN</code>	Trùng email khi tạo khách hàng

HTTP	MÃ LỖI	KHI NÀO XẢY RA
400	KYC_INVALID_TRANSITION	Chuyển KYC không hợp lệ (xem mục 6.2)
400	ACCOUNT_STATUS_TRANSITION_INVALID	Đổi trạng thái account trái máy trạng thái (vd từ closed)
400	ACCOUNT_NOT_ACTIVE	Thao tác tiền trên account không active
400	INSUFFICIENT_BALANCE	Không đủ số dư
400	CARD_STATUS_TRANSITION_INVALID	Đổi trạng thái thẻ trái máy trạng thái (từ expired)
400	TRANSACTION_NOT_REVERSIBLE	Đào giao dịch không ở trạng thái completed
400	LOAN_NOT_ACTIVE	Trả góp trên loan không active
400	LOAN_PAYMENT_ALREADY_PAID	Trả góp kỳ đã thanh toán
400	LOAN_ACCOUNT_MISSING	Loan chưa gắn account_id
400	FRAUD_ALERT_STATUS_TRANSITION_INVALID	Đổi trạng thái cảnh báo trái máy trạng thái
400	TERM_DEPOSIT_NOT_ACTIVE	Thao tác trên sổ không active
400	TERM_DEPOSIT_TOO_EARLY	Rút trước hạn khi chưa đủ 1 ngày
400	TERM_DEPOSIT_ALREADY_MATURE	Rút trước hạn khi đã tới/quá hạn
400	TERM_DEPOSIT_NOT_MATURE	Xử lý đáo hạn khi chưa tới hạn
400	TERM_DEPOSIT_NOT_MONTHLY_PAYOUT	Trả lãi tháng cho sổ không dùng payout_method=monthly
400	TERM_DEPOSIT_PAST_MATURITY	Trả lãi tháng khi đã quá hạn đáo hạn
400	TERM_DEPOSIT_NO_ACCRUAL	Trả lãi tháng khi chưa phát sinh ngày mới
400	AUTH_QUEUE_SELF_AUTHORIZATION	Checker trùng Maker (tự duyệt/tự từ chối)
400	AUTH_QUEUE_NOT_PENDING	Duyệt/từ chối yêu cầu đã quyết định rồi
400	CATEGORY_MISSING	Danh mục giao dịch cần thiết chưa seed (lỗi cấu hình môi trường)
400	GL_RULE_MISSING	Thiếu gl_posting_rules cho 1 txn_type (lỗi cấu hình môi trường)
500	INTERNAL_ERROR	Lỗi hệ thống không xác định (vd vi phạm UNIQUE constraint không được bắt buộc minh)

6. Máy trạng thái tham chiếu (State Machines)

Dùng để thiết kế test theo kỹ thuật State Transition — mỗi bảng liệt kê **toàn bộ transition hợp lệ**; bất kỳ cặp (trạng thái hiện tại → trạng thái đích) nào KHÔNG có trong bảng đều phải bị hệ thống từ chối.

6.1 `accounts.status`

`active` → `{dormant, frozen, closed}` · `dormant` → `{active, frozen, closed}` · `frozen` → `{active, dormant, closed}` · `closed` → *(terminal, không transition)*

6.2 `customers.kyc_status`

`pending` → `{verified, rejected}` **duy nhất** — không cho phép bất kỳ transition nào khác kể cả `verified` → `rejected` hay đặt lại `→ pending` .

6.3 `cards.status`

`active` → `{blocked, expired}` · `blocked` → `{active, expired}` · `expired` → *(terminal)*

6.4 `fraud_alerts.status`

`open` → `{reviewing, closed_fp, closed_confirmed}` · `reviewing` → `{closed_fp, closed_confirmed}` · `closed_fp` → / `closed_confirmed` → *(terminal)*

6.5 `loans.status`

`active` → `closed` (trả hết kỳ cuối) · `active` → `defaulted` (qua COB, quá hạn > 90 ngày) · `closed` / `defaulted` → *(terminal, không có API đổi tay)*

6.6 `term_deposits.status`

`active` → `matured` (qua `/mature`) · `active` → `withdrawn` (qua `/withdraw-early`) · `matured` / `withdrawn` → *(terminal)*

6.7 `auth_queue.status`

`pending` → `authorized` (qua `/authorize`) · `pending` → `rejected` (qua `/reject`) · `authorized` / `rejected` → *(terminal)*

6.8 Tổng hợp: 2 kiểu mô hình trạng thái trong hệ thống

Nhìn lại 7 máy trạng thái trên, toàn hệ thống chỉ rơi vào đúng 2 kiểu mô hình. Tester nên dùng bảng này để quyết định kỹ thuật test phù hợp: nhóm A cần test cả 2 chiều đi/về giữa các trạng thái sống, nhóm B chỉ cần test đúng 1 lần chuyển tiến rồi xác nhận không thể lùi/lặp lại.

ĐỐI TƯỢNG	NHÓM	CÁC TRẠNG THÁI "ĐANG SỐNG" CÓ ĐI QUA LẠI ĐƯỢC KHÔNG?	TRẠNG THÁI CUỐI (TERMINAL, HẾT ĐƯỜNG)
<code>accounts.status</code>	A — Linh hoạt + 1 ngõ cụt	Có — <code>active</code> / <code>dormant</code> / <code>frozen</code> chuyển qua lại tự do, không giới hạn số lần	<code>closed</code>
<code>cards.status</code>	A — Linh hoạt + 1 ngõ cụt	Có — <code>active</code> ↔ <code>blocked</code> chuyển qua lại tự do	<code>expired</code>
<code>customers.kyc_status</code>	B — 1 chiều tuyệt đối	Không — đúng 1 lần chuyển duy nhất từ <code>pending</code> , không có trạng thái "đang sống" nào khác để quay lại	<code>verified</code> hoặc <code>rejected</code>
<code>fraud_alerts.status</code>	B — 1 chiều tuyệt đối	Không — <code>open</code> → <code>reviewing</code> chỉ tiến, không có transition lùi về <code>open</code>	<code>closed_fp</code> hoặc <code>closed_confirmed</code>
<code>loans.status</code>	B — 1 chiều tuyệt đối	Không — chỉ 1 lần chuyển, hoàn toàn qua batch COB hoặc trả hết nợ, không có API đổi tay	<code>closed</code> hoặc <code>defaulted</code>
<code>term_deposits.status</code>	B — 1 chiều tuyệt đối	Không — chỉ 1 lần chuyển khi đáo hạn hoặc rút trước hạn	<code>matured</code> hoặc <code>withdrawn</code>
<code>auth_queue.status</code>	B — 1 chiều tuyệt đối	Không — chỉ 1 lần chuyển khi duyệt hoặc từ chối	<code>authorized</code> hoặc <code>rejected</code>

Ý nghĩa cho test design:

- Nhóm A** (`accounts` , `cards`): phải test đủ các cặp chuyển đổi qua lại giữa trạng thái sống (ví dụ `active→frozen→active→dormant→closed`), VÀ test rằng một khi đã `closed` / `expired` thì mọi transition tiếp theo đều bị từ chối (`*_STATUS_TRANSITION_INVALID`).
- Nhóm B**: không cần test chiều "quay lại" vì bản thân model không có cạnh đó — chỉ cần test (a) transition tiến đúng 1 lần thành công, và (b) gọi lại lần 2 (dù cùng giá trị hay giá trị khác) đều bị chặn. Đây chính là nhóm dễ bị bỏ sót nhất khi viết test, vì tester quen tư duy "trạng thái nào cũng sửa lại được".

7. Danh sách Test Case theo module

Ký hiệu preconditions: `[Acc-VND-1]` , `[Acc-VND-2]` , `[Acc-USD]` = các account tạo ở mục 3.3; `[Cust-A]` , `[Cust-B]` = customer.

7.1 FR-REF — Danh mục dùng chung

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-REF-001	Xem danh sách loại tài khoản	<code>GET /account-types</code>	200, có <code>id=1 Savings 2.50</code>	<code>SELECT * FROM public.account_types ORDER BY account_type_id;</code>	P3
TC-REF-002	Tạo loại tài khoản mới	<code>POST /account-types {type_name:"VIP", interest_rate:5}</code>	201	<code>SELECT * FROM public.account_types WHERE type_name='VIP';</code> → đúng 1 dòng, <code>interest_rate=5.00</code>	P3
TC-REF-003	Tạo loại tài khoản trùng tên	Gọi lại TC-REF-002 với cùng <code>type_name</code>	Lỗi (vi phạm UNIQUE, không crash server)	<code>SELECT COUNT(*) FROM public.account_types WHERE type_name='VIP';</code> → vẫn <code>=1</code> (không tăng thành 2)	P3
TC-REF-004	Xem danh sách merchant	<code>GET /merchants</code>	200, mảng (rỗng nếu chưa tạo)	<code>SELECT COUNT(*) FROM public.merchants;</code>	P3
TC-REF-005	Xem danh mục giao dịch	<code>GET /transaction-categories</code>	200, có đủ <code>Deposit/Withdrawal/Transfer/Interest</code>	<code>SELECT category_name, category_group FROM public.transaction_categories ORDER BY category_id;</code>	P2

7.2 FR-CUST — Khách hàng

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-CUST-001	Tạo khách hàng hợp lệ	<code>POST /customers</code> email chưa tồn tại	201, <code>kyc_status='pending'</code>	<code>SELECT * FROM public.customers WHERE customer_id=:id;</code>	P1
TC-CUST-002	Tạo trùng email	Lặp lại TC-CUST-001 cùng email	409 <code>CUSTOMER_EMAIL_TAKEN</code>	<code>SELECT COUNT(*) FROM public.customers WHERE email=:email;</code> → vẫn <code>=1</code>	P1
TC-CUST-003	<code>referred_by</code> không tồn tại	<code>referred_by: 999999</code>	404 <code>CUSTOMER_NOT_FOUND</code>	<code>SELECT COUNT(*) FROM public.customers WHERE referred_by=999999;</code> → <code>=0</code> (không tạo được)	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-CUST-004	Tìm kiếm theo email	<code>GET /customers?email=...</code>	200, đúng bản ghi	<code>SELECT customer_id FROM public.customers WHERE email=:email;</code>	P2
TC-CUST-005	Tìm kiếm theo kyc_status	<code>GET /customers?kyc_status=pending</code>	200, đúng danh sách	<code>SELECT customer_id FROM public.customers WHERE kyc_status='pending';</code>	P2
TC-CUST-006	Xem chi tiết không tồn tại	<code>GET /customers/999999</code>	404	<i>(không cần — xác nhận qua response là đủ)</i>	P2
TC-CUST-007	Cập nhật thông tin	<code>PATCH /customers/:id {full_name}</code>	200, đã đổi	<code>SELECT full_name, updated_at FROM public.customers WHERE customer_id=:id; → updated_at mới hơn created_at (trigger set_updated_at)</code>	P2
TC-CUST-008	KYC pending→verified	<code>PATCH /:id/kyc {kyc_status:verified}</code>	200	<code>SELECT kyc_status FROM public.customers WHERE customer_id=:id; → verified</code>	P1
TC-CUST-009	KYC verified→rejected (đã verified)	Lặp lại <code>PATCH .../kyc {rejected}</code> sau TC-CUST-008	400 <code>KYC_INVALID_TRANSITION</code>	<code>SELECT kyc_status FROM public.customers WHERE customer_id=:id; → vẫn verified , không đổi thành rejected</code>	P1
TC-CUST-010	KYC đặt lại về pending	<code>PATCH .../kyc {pending}</code> (bất kỳ trạng thái nào)	400 <code>KYC_INVALID_TRANSITION</code>	<code>SELECT kyc_status FROM public.customers WHERE customer_id=:id; → giữ nguyên giá trị trước đó</code>	P2
TC-CUST-011	Xem danh sách referrals	<code>GET /:id/referrals</code> sau khi có khách hàng khác <code>referred_by=:id</code>	200, đúng danh sách	<code>SELECT customer_id, full_name FROM public.customers WHERE referred_by=:id;</code>	P3

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-CUST-012	Xem accounts của khách hàng	GET <code>/:id/accounts</code>	200	SELECT account_id, balance FROM public.accounts WHERE customer_id=:id;	P3
TC-CUST-013	Tạo khoản vay thiếu Idempotency-Key	POST <code>/:id/loans</code> không header	400 IDEMPOTENCY_KEY_REQUIRE D	SELECT COUNT(*) FROM public.loans WHERE customer_id=:id; → =0	P1
TC-CUST-014	Tạo khoản vay lặp lại đúng key	Gọi 2 lần cùng key+body	Lần 2 trả cùng loan_id , không tạo bản ghi/giải ngân mới	SELECT COUNT(*) FROM public.loans WHERE customer_id=:id; → =1 (không phải 2); SELECT balance FROM public.accounts WHERE account_id=:acc; → chỉ +principal đúng 1 lần; SELECT COUNT(*) FROM public.idempotency_keys WHERE idempotency_key=:key; → =1	P1

7.3 FR-ACC — Tài khoản

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-ACC-001	Mở tài khoản hợp lệ	POST <code>/accounts</code> {customer_id, account_type_id:1, currency:VND}	201, balance=0.00 , account_number dạng VNxxxxxxxxxxxx	SELECT balance, currency, account_number, status FROM public.accounts WHERE account_id=:id; → balance=0.00, status='active'	P1
TC-ACC-002	account_type_id không tồn tại	account_type_id: 999	404	SELECT COUNT(*) FROM public.accounts WHERE customer_id=:cust AND account_type_id=999; → =0	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-ACC-003	<code>customer_id</code> không tồn tại	<code>customer_id: 999999</code>	404	<code>SELECT COUNT(*) FROM public.accounts WHERE customer_id=999999; → =0</code>	P2
TC-ACC-004	Xem chi tiết	<code>GET /accounts/:id</code>	200	<code>SELECT * FROM public.accounts WHERE account_id=:id;</code>	P2
TC-ACC-005	active → frozen	<code>PATCH /:id/status {frozen}</code>	200	<code>SELECT status FROM public.accounts WHERE account_id=:id; → frozen</code>	P1
TC-ACC-006	frozen → active	<code>PATCH /:id/status {active}</code>	200 (chuyển tự do, không phải chain 1 chiều)	<code>SELECT status FROM public.accounts WHERE account_id=:id; → active</code>	P1
TC-ACC-007	active → closed	<code>PATCH /:id/status {closed}</code>	200, <code>closed_at</code> được set	<code>SELECT status, closed_at FROM public.accounts WHERE account_id=:id; → closed, closed_at IS NOT NULL</code>	P1
TC-ACC-008	closed → active (mở lại)	<code>PATCH /:id/status {active}</code> sau TC-ACC-007	400 <code>ACCOUNT_STATUS_TRANSITION_INVALID</code>	<code>SELECT status FROM public.accounts WHERE account_id=:id; → vẫn closed</code>	P1
TC-ACC-009	Lịch sử số dư	Thực hiện 3 giao dịch rồi <code>GET /:id/balance-history</code>	200, <code>running_balance</code> khớp phép cộng dồn tay	<code>SELECT txn_type, amount, txn_timestamp FROM public.transactions WHERE account_id=:id OR related_account_id=:id ORDER BY txn_timestamp; → cộng dồn tay đối chiếu running_balance</code>	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-ACC-010	Danh sách giao dịch có lọc	<code>GET /:id/transactions?txn_type=deposit</code>	200, chỉ trả đúng loại	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id AND txn_type='deposit';</code> → khớp <code>meta.total</code>	P2
TC-ACC-011	Danh sách sổ tiết kiệm theo account	<code>GET /:id/term-deposits</code>	200	<code>SELECT term_deposit_id, status FROM public.term_deposits WHERE account_id=:id;</code>	P3

7.4 FR-CARD — Thẻ

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-CARD-001	Phát hành thẻ cho account active	<code>POST /accounts/:id/cards {card_type:debit, expiry_date}</code>	201	<code>SELECT * FROM public.cards WHERE account_id=:id;</code> → có dòng mới, <code>status='active'</code>	P2
TC-CARD-002	Phát hành thẻ cho account frozen/closed	Đổi account sang <code>frozen</code> trước, rồi phát hành	400 <code>ACCOUNT_NOT_ACTIVE</code>	<code>SELECT COUNT(*) FROM public.cards WHERE account_id=:id;</code> → không tăng	P2
TC-CARD-003	Xem chi tiết thẻ	<code>GET /cards/:id</code>	200	<code>SELECT * FROM public.cards WHERE card_id=:id;</code>	P3
TC-CARD-004	active→blocked→active→expire d	3 lần <code>PATCH /:id/status</code> liên tiếp	Mỗi bước 200	<code>SELECT status FROM public.cards WHERE card_id=:id;</code> sau mỗi bước	P2
TC-CARD-005	expired → bất kỳ	<code>PATCH /:id/status {active}</code> sau khi đã <code>expired</code>	400 <code>CARD_STATUS_TRANSITION_INVALID</code>	<code>SELECT status FROM public.cards WHERE card_id=:id;</code> → vẫn <code>expired</code>	P2

7.5 FR-TXN — Giao dịch (module lỗi, ưu tiên cao)

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TXN-001	Deposit hợp lệ	<code>POST /transactions</code> <code>{account_id,</code> <code>txn_type:deposit,</code> <code>amount, category_id}</code> + Idempotency-Key	201, balance +amount	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:id; + SELECT</code> <code>* FROM</code> <code>public.transactions</code> <code>WHERE</code> <code>transaction_id=:txn;</code>	P1
TC-TXN-002	Withdrawal đủ số dư	<code>txn_type:withdrawal</code> , amount ≤ balance	201, balance -amount	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:id;</code>	P1
TC-TXN-003	Withdrawal không đủ số dư	amount > balance	400 <code>INSUFFICIENT_BALANCE</code>	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:id; → không</code> đổi	P1
TC-TXN-004	Transfer hợp lệ cùng currency	<code>[Acc-VND-1]→[Acc-VND-2]</code> , <code>txn_type:transfer</code>	201, cả 2 balance đổi đúng	<code>SELECT account_id,</code> <code>balance FROM</code> <code>public.accounts WHERE</code> <code>account_id IN (:a1,:a2);</code>	P1
TC-TXN-005	Transfer thiếu <code>related_account_id</code>	<code>txn_type:transfer</code> không kèm field	422 <code>VALIDATION_ERROR</code>	<i>(bị chặn trước khi ghi DB — xác nhận qua response là đủ)</i>	P1
TC-TXN-006	<code>related_account_id</code> trùng <code>account_id</code>	Tự transfer cho chính mình	422	<i>(không ghi được — xác nhận qua response là đủ)</i>	P2
TC-TXN-007	Có <code>related_account_id</code> nhưng không phải transfer	<code>txn_type:deposit</code> kèm <code>related_account_id</code>	422	<i>(không ghi được — xác nhận qua response là đủ)</i>	P2
TC-TXN-008	Currency lệch tài khoản	<code>[Acc-VND-1]</code> nhưng <code>currency:USD</code>	422 <code>VALIDATION_ERROR</code> (đã vả — không âm thầm sai số)	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:id; → không</code> đổi (bảng chứng không bị cộng/trừ sai)	P1
TC-TXN-009	Transfer khác currency qua endpoint thường	<code>[Acc-VND-1]→[Acc-USD]</code>	422, gợi ý dùng <code>/fx-</code> <code>transfers</code>	<code>SELECT balance, currency</code> <code>FROM public.accounts</code> <code>WHERE account_id IN</code> <code>(:vnd,:usd); → cả 2 không</code> đổi	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TXN-010	Giao dịch trên account không active	Đổi account sang frozen , thử deposit	400 ACCOUNT_NOT_ACTIVE	<code>SELECT balance FROM public.accounts WHERE account_id=:id; → không đổi</code>	P1
TC-TXN-011	category_id không tồn tại	category_id: 999	404	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → không tăng</code> (bảng chứng 404 không phải kiểu "báo lỗi cho có" trong khi vẫn âm thầm ghi DB)	P2
TC-TXN-012	merchant_id không tồn tại	merchant_id: 999	404	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → không tăng</code>	P3
TC-TXN-013	card_id không thuộc account	Thẻ của account khác	422	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → không tăng</code>	P3
TC-TXN-014	Thiếu Idempotency-Key	POST /transactions không header	400 IDEMPOTENCY_KEY_REQUIRED	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → không tăng</code>	P1
TC-TXN-015	Lặp lại đúng key+body	Gọi 2 lần	Lần 2 trả cache, balance không đổi thêm	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → chỉ +1 dòng dù gọi 2 lần;</code> <code>SELECT balance FROM public.accounts WHERE account_id=:id; → chỉ +amount đúng 1 lần</code>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TXN-016	Cùng key khác body	Đổi <code>amount</code> giữ nguyên key	409 <code>IDEMPOTENCY_KEY_CONFLICT</code>	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:id; → không có dòng mới cho lần gọi thứ 2</code>	P1
TC-TXN-017	Vượt ngưỡng gian lận	<code>amount > FRAUD_AMOUNT_THRESHOLD</code>	201 + tự sinh <code>fraud_alerts</code> (kiểm <code>GET /fraud-alerts</code>)	<code>SELECT * FROM public.fraud_alerts WHERE transaction_id=:txn; → có dòng, alert_type='LARGE_AMOUNT'</code>	P1
TC-TXN-018	Đào giao dịch hợp lệ	<code>POST /:id/reverse</code> trên giao dịch <code>completed</code>	201, balance hoàn nguyên, GL 2 dòng đảo Nợ/Có	<code>SELECT status FROM public.transactions WHERE transaction_id=:id; → reversed; SELECT balance FROM public.accounts WHERE account_id=:acc; → về đúng giá trị trước giao dịch gốc</code>	P1
TC-TXN-019	Đào giao dịch đã đảo rồi	Gọi <code>/reverse</code> lần 2	400 <code>TRANSACTION_NOT_REVERSIBLE</code>	<code>SELECT COUNT(*) FROM public.transactions WHERE account_id=:acc AND description LIKE 'Reversal of transaction #%'; → vẫn =1 cho giao dịch gốc :id (không đảo 2 lần)</code>	P2
TC-TXN-020	Đào giao dịch không tồn tại	<code>POST /999999/reverse</code>	404	<code>SELECT COUNT(*) FROM public.transactions WHERE description LIKE 'Reversal of transaction #999999%'; → = 0 (không có bản ghi đảo nào được tạo cho 1 giao dịch gốc không tồn tại)</code>	P3

7.6 FR-TD — Tiền gửi có kỳ hạn (module khó nhất — nhiều case)

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TD-001	Mở sổ hợp lệ (lãi đơn, trả cuối kỳ)	<code>POST /term-deposits</code> { <code>account_id</code> , <code>principal_amount</code> , <code>interest_rate</code> , <code>term_months</code> } + key	201, account balance -principal	<code>SELECT * FROM</code> <code>public.term_deposits</code> <code>WHERE</code> <code>term_deposit_id=:td;</code> + <code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:acc;</code>	P1
TC-TD-002	Mở sổ thiếu Idempotency-Key	Không header	400	<code>SELECT COUNT(*) FROM</code> <code>public.term_deposits</code> <code>WHERE account_id=:acc;</code> → không tăng	P1
TC-TD-003	Mở sổ trên account không active	Account <code>frozen</code>	400 <code>ACCOUNT_NOT_ACTIVE</code>	<code>SELECT COUNT(*) FROM</code> <code>public.term_deposits</code> <code>WHERE account_id=:acc;</code> → không tăng	P2
TC-TD-004	Mở sổ không đủ số dư	<code>principal_amount</code> > balance	400 <code>INSUFFICIENT_BALANCE</code>	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:acc;</code> → không đổi	P1
TC-TD-005	Xem lãi dự thu (lãi đơn)	<code>GET /:id/accrued-</code> <code>interest</code> giữa kỳ	200, <code>accrued_interest</code> = <code>Pxr%xngày/365</code> (đối chiếu tay)	<code>SELECT principal_amount,</code> <code>interest_rate,</code> <code>start_date,</code> <code>day_count_convention</code> <code>FROM</code> <code>public.term_deposits</code> <code>WHERE</code> <code>term_deposit_id=:id;</code> → dùng để tự tính lại và so khớp	P1
TC-TD-006	Xem lãi dự thu (lãi kép)	Sổ <code>interest_method=compound</code> d	200, giá trị nội suy tuyến tính theo tỉ lệ ngày	<code>SELECT interest_method,</code> <code>term_months FROM</code> <code>public.term_deposits</code> <code>WHERE</code> <code>term_deposit_id=:id;</code>	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TD-007	Rút trước hạn khi chưa đủ 1 ngày	<code>start_date</code> = hôm nay, gọi <code>/withdraw-early</code> ngay	400 <code>TERM_DEPOSIT_TOO_EARLY</code>	<code>SELECT status FROM public.term_deposits WHERE term_deposit_id=:id; → vẫn active</code>	P2
TC-TD-008	Rút trước hạn hợp lệ (case khó #1)	Số lãi suất cam kết cao (vd 7%), rút giữa kỳ	201, lãi nhận chỉ theo <code>early_withdrawal_rate</code> (vd 0.20%), không phải 7%	<code>SELECT status, closed_date FROM public.term_deposits WHERE term_deposit_id=:id; → withdrawn; SELECT interest_amount FROM public.term_deposit_postings WHERE term_deposit_id=:id AND posting_type='early_withdrawal_settlement'; → khớp công thức lãi KHÔNG kỳ hạn, không phải 7%</code>	P1
TC-TD-009	Rút trước hạn khi đã tới/qua hạn	<code>maturity_date</code> ≤ hôm nay	400 <code>TERM_DEPOSIT_ALREADY_MATURE</code>	<code>SELECT status FROM public.term_deposits WHERE term_deposit_id=:id; → vẫn active</code>	P2
TC-TD-010	Rút trước hạn sổ không active	Số đã <code>matured</code> / <code>withdrawn</code>	400 <code>TERM_DEPOSIT_NOT_ACTIVE</code>	<code>SELECT status FROM public.term_deposits WHERE term_deposit_id=:id; → không đổi thêm</code>	P2
TC-TD-011	Đáo hạn khi chưa tới hạn	<code>maturity_date</code> > hôm nay	400 <code>TERM_DEPOSIT_NOT_MATURE</code>	<code>SELECT status FROM public.term_deposits WHERE term_deposit_id=:id; → vẫn active</code>	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TD-012	Đáo hạn, auto_renewal=false	Số đã tới hạn	201, account + gốc+lãi, GL tách Nợ 2100(gốc)+Nợ 5000(lãi)/Có 2000	<pre>SELECT status FROM public.term_deposits WHERE term_deposit_id=:id; → matured; SELECT gl_account_id, entry_side, amount FROM public.gl_entries WHERE transaction_id=(SELECT transaction_id FROM public.term_deposit_postings WHERE term_deposit_id=:id ORDER BY posting_id DESC LIMIT 1); → 3 dòng đúng như mô tả</pre>	P1
TC-TD-013	Đáo hạn, auto_renewal=true (case khó #2)	Số đã tới hạn, auto_renewal=true	201, tạo sổ mới principal=gốc+lãi, renewed_from_id đúng, account balance không đổi	<pre>SELECT term_deposit_id, principal_amount, renewed_from_id FROM public.term_deposits WHERE renewed_from_id=:old_id; → sổ mới đúng gốc; SELECT balance FROM public.accounts WHERE account_id=:acc; → không đổi so với trước khi đáo hạn</pre>	P1
TC-TD-014	Đáo hạn lãi kép	interest_method=compound, term=12 tháng	Lãi = $P \times ((1+r\%/12)^{12}-1)$ (đối chiếu tay)	<pre>SELECT interest_amount FROM public.term_deposit_postings WHERE term_deposit_id=:id AND posting_type='maturity_settlement';</pre>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TD-015	Trả lãi tháng lần đầu	Số <code>payout_method=monthly</code>	201, account +lãi đúng số ngày kể từ <code>start_date</code>	<code>SELECT period_from, period_to, days, interest_amount FROM public.term_deposit_postings WHERE term_deposit_id=:id;</code>	P1
TC-TD-016	Trả lãi tháng 2 lần liên tiếp cùng ngày	Gọi <code>/interest-postings/monthly</code> 2 lần liên	Lần 2: 400 <code>TERM_DEPOSIT_NO_ACCRUAL</code>	<code>SELECT COUNT(*) FROM public.term_deposit_postings WHERE term_deposit_id=:id; → chỉ tăng 1, không phải 2</code>	P2
TC-TD-017	Trả lãi tháng cho sổ <code>payout_method=maturity</code>	Gọi trên sổ sai loại	400 <code>TERM_DEPOSIT_NOT_MONTHLY_PAYOUT</code>	<code>SELECT COUNT(*) FROM public.term_deposit_postings WHERE term_deposit_id=:id; → =0</code>	P2
TC-TD-018	Trả lãi tháng khi đã quá hạn	<code>maturity_date</code> < hôm nay	400 <code>TERM_DEPOSIT_PAST_MATURITY</code>	<code>SELECT COUNT(*) FROM public.term_deposit_postings WHERE term_deposit_id=:id AND posting_type='monthly_payout'; → không tăng</code>	P2
TC-TD-019	Xem lịch sử ghi nhận lãi	<code>GET /:id/postings</code> sau vài thao tác	200, mỗi dòng có <code>transaction_id</code> thật, tra được qua <code>/gl/transactions/:id/entries</code>	<code>SELECT posting_id, transaction_id FROM public.term_deposit_postings WHERE term_deposit_id=:id; → mọi transaction_id đều IN (SELECT transaction_id FROM public.transactions)</code>	P2
TC-TD-020	Danh sách sổ theo account	<code>GET /accounts/:id/term-deposits</code>	200	<code>SELECT term_deposit_id FROM public.term_deposits WHERE account_id=:id;</code>	P3

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-TD-021	Mở sổ thiếu trường bắt buộc	Thiếu <code>interest_rate</code>	422	<i>(bị chặn trước khi ghi DB — xác nhận qua response là đủ)</i>	P2

7.7 FR-LOAN — Cho vay

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-LOAN-001	Tạo khoản vay hợp lệ	<code>POST</code> <code>/customers/:id/loans</code> + key	201, N kỳ trả góp, Σ <code>principal_component</code> = <code>principal_amount</code> , account + principal	<code>SELECT SUM(principal_component) FROM public.loan_payments WHERE loan_id=:id; → bằng principal_amount; SELECT balance FROM public.accounts WHERE account_id=:acc;</code>	P1
TC-LOAN-002	<code>account_id</code> không thuộc customer	Account của khách hàng khác	422	<i>(bị chặn trước khi ghi — xác nhận qua response là đủ)</i>	P2
TC-LOAN-003	Account không active	Account <code>frozen</code>	400 <code>ACCOUNT_NOT_ACTIVE</code>	<code>SELECT COUNT(*) FROM public.loans WHERE account_id=:acc; → không tăng</code>	P2
TC-LOAN-004	Thiếu Idempotency-Key	Không header	400	<code>SELECT COUNT(*) FROM public.loans WHERE customer_id=:cust; → không tăng</code>	P1
TC-LOAN-005	Lặp lại đúng key	Gọi 2 lần	Cùng <code>loan_id</code> , không giải ngân 2 lần (đối chiếu balance)	<code>SELECT COUNT(*) FROM public.loans WHERE customer_id=:cust; → +1 dù gọi 2 lần</code>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-LOAN-006	Trả góp đúng <code>amount_due</code>	POST /:id/payments/:pid/pay + key	200, GL tách đúng gốc/lãi theo lịch	SELECT paid_date, amount_paid, transaction_id FROM public.loan_payments WHERE payment_id=:pid; + SELECT gl_account_id, entry_side, amount FROM public.gl_entries WHERE transaction_id=(SELECT transaction_id FROM public.loan_payments WHERE payment_id=:pid);	P1
TC-LOAN-007	Trả góp không đủ số dư	Account balance < <code>amount_paid</code>	400 <code>INSUFFICIENT_BALANCE</code>	SELECT paid_date FROM public.loan_payments WHERE payment_id=:pid; → vẫn NULL	P1
TC-LOAN-008	Trả góp kỳ đã trả	Gọi lại cùng <code>payment_id</code> khác key	400 <code>LOAN_PAYMENT_ALREADY_PAID</code>	SELECT amount_paid FROM public.loan_payments WHERE payment_id=:pid; → không đổi so với lần trả đầu	P2
TC-LOAN-009	Trả hết kỳ cuối	Trả đủ N/N kỳ	<code>loans.status</code> → <code>closed</code>	SELECT status FROM public.loans WHERE loan_id=:id; → <code>closed</code> ; SELECT COUNT(*) FROM public.loan_payments WHERE loan_id=:id AND paid_date IS NULL; → <code>=0</code>	P1
TC-LOAN-010	Trả góp trên loan không active	Loan đã <code>closed</code> / <code>defaulted</code>	400 <code>LOAN_NOT_ACTIVE</code>	SELECT status FROM public.loans WHERE loan_id=:id; → không đổi	P2
TC-LOAN-011	Trả góp thiếu Idempotency-Key	Không header	400	SELECT paid_date FROM public.loan_payments WHERE payment_id=:pid; → vẫn NULL	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-LOAN-012	<code>payment_id</code> không thuộc <code>loan_id</code>	Trộn ID sai	404 <code>LOAN_PAYMENT_NOT_FOUND</code>	<pre>SELECT amount_paid, paid_date, transaction_id FROM public.loan_payments WHERE payment_id=:paymentId; → vẫn amount_paid / paid_date / transaction_id như trước (không bị áp nhầm khoản thanh toán từ 1 loan khác vào kỳ trả nợ này — bảng loan_payments không có cột status riêng, trạng thái "đã trả" được suy ra từ paid_date IS NOT NULL)</pre>	P3

7.8 FR-FRAUD — Cảnh báo gian lận

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-FRAUD-001	Giao dịch vượt ngưỡng	<code>amount = threshold + 1</code>	Tự tạo <code>fraud_alerts</code> , <code>risk_score = round((amount/threshold) × 50)</code>	<pre>SELECT risk_score, alert_type FROM public.fraud_alerts WHERE transaction_id=:txn;</pre>	P1
TC-FRAUD-002	Giao dịch đúng ngưỡng (boundary)	<code>amount = threshold</code> (không hơn)	Không tạo alert (điều kiện là <code>></code> , không phải <code>≥</code>)	<pre>SELECT COUNT(*) FROM public.fraud_alerts WHERE transaction_id=:txn; → =0</pre>	P1
TC-FRAUD-003	Giao dịch rất lớn	<code>amount = threshold × 5</code>	<code>risk_score</code> bị chặn ở tối đa 100	<pre>SELECT risk_score FROM public.fraud_alerts WHERE transaction_id=:txn; → =100</pre>	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-FRAUD-004	Tạo cảnh báo thủ công	POST /fraud-alerts	201	SELECT * FROM public.fraud_alerts WHERE alert_id=:id;	P2
TC-FRAUD-005	open → reviewing	PATCH /:id/status {reviewing}	200	SELECT status FROM public.fraud_alerts WHERE alert_id=:id; → reviewing	P2
TC-FRAUD-006	open → closed_fp (bỏ qua reviewing)	PATCH /:id/status {closed_fp}	200 (transition thẳng hợp lệ)	SELECT status FROM public.fraud_alerts WHERE alert_id=:id; → closed_fp	P2
TC-FRAUD-007	reviewing → open (lùi)	PATCH /:id/status {open}	400 FRAUD_ALERT_STATUS_TRANSITION_INVALID	SELECT status FROM public.fraud_alerts WHERE alert_id=:id; → vẫn reviewing	P2
TC-FRAUD-008	closed_fp → bất kỳ	Sau khi đã closed_fp	400 (terminal)	SELECT status FROM public.fraud_alerts WHERE alert_id=:id; → vẫn closed_fp	P2

7.9 FR-FX — Tỷ giá & Quy đổi ngoại tệ

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-FX-001	Tra cứu tỷ giá as-of đúng ngày	GET /exchange-rates? from=USD&to=VND&date=<ngày seed>	200	SELECT * FROM public.exchange_rates WHERE currency_from='USD' AND currency_to='VND' ORDER BY rate_date DESC;	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-FX-002	Tra cứu as-of ngày sau (không có bản ghi mới hơn)	<code>date</code> = hôm sau ngày seed	200, trả bản ghi gần nhất trước đó	<pre>SELECT rate_date FROM public.exchange_rates WHERE currency_from='USD' AND currency_to='VND' AND rate_date <= :date ORDER BY rate_date DESC LIMIT 1;</pre>	P2
TC-FX-003	Tra cứu tỷ giá không tồn tại	Cập tiền tệ chưa từng seed	404 <code>EXCHANGE_RATE_NOT_FOUND</code>	<pre>SELECT COUNT(*) FROM public.exchange_rates WHERE currency_from=:cf AND currency_to=:ct; → =0</pre>	P2
TC-FX-004	Tạo tỷ giá trùng ngày/cặp	Tạo lại đúng (<code>from,to,date</code>) với <code>rate</code> khác	201, <code>rate</code> được ghi đè (UPSERT)	<pre>SELECT rate FROM public.exchange_rates WHERE currency_from=:cf AND currency_to=:ct AND rate_date=:d; → giá trị mới nhất, chỉ 1 dòng duy nhất (không trùng lặp)</pre>	P3
TC-FX-005	Quy đổi USD→VND hợp lệ (khớp tỷ giá đã seed ở mục 3.3)	<pre>POST /fx-transfers {from:Acc-USD, to:Acc-VND, from_amount} + key</pre>	201, <code>to_amount</code> = <code>from_amount × rate</code> (đối chiếu tay, vd 200 USD × 25000 = 5.000.000 VND)	<pre>SELECT balance, currency FROM public.accounts WHERE account_id IN (:usd,:vnd); + SELECT gl_account_id, entry_side, amount FROM public.gl_entries WHERE transaction_id=(SELECT transaction_id FROM public.transactions WHERE account_id=:vnd ORDER BY transaction_id DESC LIMIT 1); → 1 cặp Nợ/Có cùng mã 2000</pre>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-FX-006	Quy đổi VND→USD hợp lệ (chiều ngược)	Cần seed thêm <code>exchange_rates</code> { <code>currency_from</code> :VND, <code>currency_to</code> :USD, <code>rate</code> :0.00004, <code>rate_date</code> } (tra cứu theo đúng chiều <code>from_account.currency</code> → <code>to_account.currency</code> , không tự đảo ngược tỷ giá đã seed chiều kia), sau đó <code>POST</code> <code>/fx-transfers</code> { <code>from</code> :Acc- VND, <code>to</code> :Acc-USD, <code>from_amount</code> }	201, <code>to_amount</code> = <code>from_amount</code> × <code>rate</code>	<code>SELECT balance, currency</code> <code>FROM public.accounts</code> <code>WHERE account_id IN</code> (:vnd,:usd);	P1
TC-FX-007	Quy đổi 2 account cùng currency	[Acc-VND-1]→[Acc-VND-2]	422, gợi ý dùng transfer thường	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id IN (:a1,:a2);</code> → cả 2 không đổi	P1
TC-FX-008	Quy đổi cặp không có chân VND	Mở thêm 1 account <code>currency</code> :EUR , thử <code>POST</code> <code>/fx-transfers</code> { <code>from</code> :Acc- USD, <code>to</code> :Acc-EUR, <code>from_amount</code> }	422, không hỗ trợ quy đổi ngoại tệ ↔ ngoại tệ trực tiếp	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id IN</code> (:usd,:eur); → cả 2 không đổi	P1
TC-FX-009	Tài khoản nguồn không đủ số dư	<code>from_amount</code> > balance	400 <code>INSUFFICIENT_BALANCE</code>	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:from;</code> → không đổi	P2
TC-FX-010	Thiếu Idempotency-Key	Không header	400	<code>SELECT COUNT(*) FROM</code> <code>public.transactions</code> <code>WHERE account_id=:from</code> <code>OR account_id=:to;</code> → không tăng	P2
TC-FX-011	Quy đổi khi chưa seed tỷ giá	Ngày quy đổi không có <code>exchange_rates</code> phù hợp	404 <code>EXCHANGE_RATE_NOT_FOUND</code>	<code>SELECT balance FROM</code> <code>public.accounts WHERE</code> <code>account_id=:from;</code> → không đổi	P2

7.10 FR-GL — Sổ cái kế toán kép

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-GL-001	Xem Chart of Accounts	<code>GET /gl/accounts</code>	200, đủ 7 tài khoản đúng mục 3.3 SRS	<code>SELECT code, name, account_class, normal_balance FROM public.gl_accounts ORDER BY code;</code>	P2
TC-GL-002	Bảng cân đối thử sau mọi thao tác	<code>GET /gl/trial-balance</code> sau khi chạy bất kỳ nhóm test nào ở trên	<code>balanced: true , total_debit == total_credit</code>	<code>SELECT entry_side, SUM(amount) FROM public.gl_entries GROUP BY entry_side; → 2 tổng phải bằng nhau tuyệt đối</code>	P1 — chạy lại liên tục xuyên suốt bộ test
TC-GL-003	Sổ cái theo tài khoản kế toán	<code>GET /gl/accounts/2000/entries</code>	200, danh sách bút toán	<code>SELECT * FROM public.gl_entries e JOIN public.gl_accounts a ON a.gl_account_id=e.gl_account_id WHERE a.code='2000';</code>	P2
TC-GL-004	Sổ cái theo mã không tồn tại	<code>GET /gl/accounts/9999/entries</code>	404	<i>(không cần — đây là GET, không có đường ghi dữ liệu nào để kiểm tra)</i>	P3
TC-GL-005	Bút toán theo transaction	<code>GET /gl/transactions/:id/entries</code> sau TC-TXN-001	200, đúng 2 dòng Nợ=Có=amount	<code>SELECT entry_side, amount FROM public.gl_entries WHERE transaction_id=:id; → 2 dòng, amount bằng nhau</code>	P1
TC-GL-006	Bút toán đảo giao dịch	Sau TC-TXN-018 (reverse)	2 dòng mới có <code>entry_side</code> đảo ngược so với 2 dòng gốc	<code>SELECT gl_account_id, entry_side FROM public.gl_entries WHERE transaction_id=:reversal_txn; so với ...WHERE transaction_id=:original_txn; → cùng gl_account_id, entry_side ngược nhau</code>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-GL-007	Bút toán tách gốc/lãi (loan payment)	Sau TC-LOAN-006	3 dòng: Nợ 2000 (tổng) / Có 1100 (gốc) + Có 4000 (lãi)	<pre>SELECT a.code, e.entry_side, e.amount FROM public.gl_entries e JOIN public.gl_accounts a ON a.gl_account_id=e.gl_acc ount_id WHERE e.transaction_id=:txn; → đúng 3 dòng như mô tả</pre>	P1

7.11 FR-AUTH — Đăng nhập, RBAC & Kiểm soát kép (bảo mật — ưu tiên rất cao)

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-AUTH-001	Đăng nhập đúng	<code>POST /auth/login {teller1, Demo@123}</code>	200, có <code>token</code> , <code>user</code> KHÔNG chứa <code>password_hash</code>	<pre>SELECT username, role FROM public.app_users WHERE username='teller1';</pre>	P1
TC-AUTH-002	Đăng nhập sai mật khẩu	<code>password: sai</code>	401 <code>UNAUTHORIZED</code>	(không cần — không ghi gì)	P1
TC-AUTH-003	Đăng nhập username không tồn tại	<code>username: khongtontai</code>	401 (thông báo giống hệt TC-AUTH-002, không lộ user có tồn tại hay không)	<pre>SELECT COUNT(*) FROM public.app_users WHERE username='khongtontai'; → =0 (xác nhận đúng là user không tồn tại)</pre>	P1
TC-AUTH-004	<code>/auth/me</code> không có token	<code>GET /auth/me</code> không header	401	(không cần — đây là <code>GET</code> , không có đường ghi dữ liệu nào để kiểm tra)	P1
TC-AUTH-005	<code>/auth/me</code> token hỏng/hết hạn	Token giả hoặc sửa 1 ký tự	401	(không cần — đây là <code>GET</code> , không có đường ghi dữ liệu nào để kiểm tra)	P2
TC-AUTH-006	Enqueue không có token	<code>POST /auth-queue</code> không <code>Authorization</code>	401	<pre>SELECT COUNT(*) FROM public.auth_queue WHERE input_at > now() - interval '1 minute'; → không có dòng mới</pre>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-AUTH-007	Enqueue bằng role checker	Login supervisor1 , gọi enqueue	403 FORBIDDEN	<code>SELECT COUNT(*) FROM public.auth_queue WHERE maker_id=(SELECT user_id FROM public.app_users WHERE username='supervisor1'); → =0</code>	P1
TC-AUTH-008	Enqueue payload sai schema	operation_type=CREATE_TRANSACTION , payload thiếu account_id	422 ngay lúc nộp (không đợi tới lúc duyệt)	<code>SELECT COUNT(*) FROM public.auth_queue WHERE queue_id=:id; → =0 (chưa từng được tạo)</code>	P1
TC-AUTH-009	Tự duyệt lệnh của chính mình	Login teller1 , enqueue rồi cũng teller1 authorize	400 AUTH_QUEUE_SELF_AUTHORIZATION	<code>SELECT status, checker_id FROM public.auth_queue WHERE queue_id=:id; → vẫn pending, checker_id IS NULL</code>	P1
TC-AUTH-010	Duyệt bởi role sai (không phải người nộp)	teller1 enqueue, teller2 (role maker, không phải checker) authorize	403 FORBIDDEN (khác lỗi TC-AUTH-009 dù cùng "không phải checker")	<code>SELECT status FROM public.auth_queue WHERE queue_id=:id; → vẫn pending</code>	P1
TC-AUTH-011	Duyệt hợp lệ	teller1 enqueue, supervisor1 authorize	200, status=authorized , result có dữ liệu, tiền đã di chuyển thật	<code>SELECT status, checker_id, result FROM public.auth_queue WHERE queue_id=:id; → authorized, result khớp transaction_id thật đã tạo</code>	P1
TC-AUTH-012	Duyệt lại lần 2	Gọi /authorize lần nữa trên queue đã authorized	400 AUTH_QUEUE_NOT_PENDING	<code>SELECT decided_at FROM public.auth_queue WHERE queue_id=:id; → không đổi so với lần duyệt đầu</code>	P1

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-AUTH-013	Từ chối hợp lệ	<code>supervisor1</code> reject kèm <code>reason</code>	200, <code>status=rejected</code> , tiền không di chuyển	<code>SELECT status, reject_reason FROM public.auth_queue WHERE queue_id=:id; → rejected, có reject_reason</code>	P1
TC-AUTH-014	Từ chối thiếu <code>reason</code>	Body không có <code>reason</code>	422	<code>SELECT status FROM public.auth_queue WHERE queue_id=:id; → vẫn pending</code>	P2
TC-AUTH-015	Từ chối bởi chính maker	<code>teller1</code> tự reject lệnh của mình	400 <code>AUTH_QUEUE_SELF_AUTHORIZATION</code>	<code>SELECT status FROM public.auth_queue WHERE queue_id=:id; → vẫn pending</code>	P2
TC-AUTH-016	Executor lỗi sau khi claim	Enqueue payload hợp lệ về schema nhưng sẽ fail ở service (vd <code>INSUFFICIENT_BALANCE</code>), rồi authorize	Queue tự hoàn tác về <code>pending</code> (không kẹt ở <code>authorized</code> sai sự thật), lỗi service được trả về cho checker	<code>SELECT status, checker_id FROM public.auth_queue WHERE queue_id=:id; → pending, checker_id IS NULL (đã hoàn tác, không kẹt ở authorized)</code>	P1
TC-AUTH-017	Lọc hàng đợi theo trạng thái	<code>GET /auth-queue?status=pending</code>	200, chỉ trả đúng trạng thái	<code>SELECT COUNT(*) FROM public.auth_queue WHERE status='pending'; → khớp meta.total</code>	P2
TC-AUTH-018	Danh sách user không lộ mật khẩu	<code>GET /users</code>	200, không có trường <code>password_hash</code> trong bất kỳ bản ghi nào	<code>SELECT COUNT(*) FROM public.app_users; → đối chiếu số lượng bản ghi (5), không đối chiếu password_hash (API không trả trường này)</code>	P1

7.12 FR-COB — Batch cuối ngày (test job / scheduled job)

"Test job" trong hệ thống này nghĩa là gì? Toàn hệ thống chỉ có đúng 1 job chạy nền theo lịch: **COB (Close of Business)** — bản mô phỏng batch cuối ngày mà lỗi ngân hàng thật (T24, Flexcube...) luôn chạy tự động mỗi đêm sau giờ giao dịch. Mục này giải thích job chạy như thế nào và cách kiểm thử nó, để trả lời được câu hỏi phỏng vấn kiểu "mô tả cách bạn test 1 batch job/scheduled job".

Job này làm gì (`src/services/cobService.ts` , hàm `run(asOfDate?)`) — chạy tuần tự 3 bước, mỗi bước tự truy vấn "cái gì ĐANG đến hạn tính đến ngày `asOf` " rồi xử lý hàng loạt bằng đúng service nghiệp vụ đã có (không viết lại business rule riêng cho batch):

1. Sổ tiết kiệm có `maturity_date <= asOf` và còn `active` → tự gọi `termDepositService.mature()` .
2. Khoản vay có kỳ trả nợ quá hạn `> 90` ngày → chuyển `loans.status = 'defaulted'` (chuẩn Basel).
3. Thẻ đang `active` / `blocked` có `expiry_date <= asOf` → chuyển `cards.status = 'expired'` .

2 đường trigger, dùng chung đúng 1 hàm (đây là điểm quan trọng nhất khi test — sửa/test 1 lần là chắc chắn cả 2 đường đều đúng, không lệch nhau):

- **Tự động:** `node-cron` chạy theo lịch cấu hình ở biến môi trường `COB_CRON_SCHEDULE` (mặc định `0 0 * * *` — 0h mỗi ngày), khai báo ở `src/jobs/cobScheduler.ts` .
- **Thủ công:** `POST /cob/run` (body tùy chọn `{ as_of_date }`) — dùng để QA/demo trigger ngay lập tức, không phải đợi tới nửa đêm.

4 đặc tính bắt buộc phải kiểm tra khi test bất kỳ batch/scheduled job nào (áp dụng cụ thể vào COB):

ĐẶC TÍNH CẦN TEST	COB LÀM ĐÚNG NHƯ THẾ NÀO	CÁCH VERIFY
Idempotent — chạy lại nhiều lần không xử lý trùng	Mỗi bước tự query "đang active/chưa xử lý VÀ đã đến hạn" tại thời điểm chạy — không dựa vào cờ "đã chạy hôm nay chưa", nên bản ghi đã xử lý xong sẽ tự động không xuất hiện lại ở lần chạy sau	TC-COB-009: chạy <code>/cob/run</code> 2 lần liên tiếp, lần 2 phải ra số liệu rỗng cho phần đã xử lý ở lần 1
Resilient theo từng bản ghi — 1 lỗi không sập cả batch	Vòng lặp đảo hạn sổ tiết kiệm có <code>try/catch</code> riêng từng sổ (<code>cobService.ts:50-62</code>); lỗi bị gom vào <code>matured_errors</code> , các sổ khác vẫn xử lý tiếp	TC-COB-006: xem <code>details.matured_errors</code> trong 1 lần chạy có lỗi cục bộ
Audit trail đầy đủ — biết chính xác job đã làm gì, khi nào	Mỗi lần chạy ghi 1 dòng vào <code>cob_runs</code> (ngày chạy, số liệu tổng hợp, <code>status</code> , <code>details</code>) JSON liệt kê từng ID đã xử lý)	TC-COB-005, TC-COB-006
Test được không cần chờ thời gian thực trôi qua	Tham số <code>as_of_date</code> cho phép "giả lập" job chạy vào 1 ngày trong tương lai — kỹ thuật bắt buộc phải có khi test bất kỳ job nào phụ thuộc ngày tháng, nếu không QA phải chờ thật sự tới ngày đó mới test được	Toàn bộ TC-COB-002 → 004, 008 đều dùng kỹ thuật này thay vì chờ ngày thật

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-COB-001	Chạy COB không có gì đến hạn	<code>POST /cob/run</code> khi không có sổ/vay/thẻ đến hạn	201, <code>term_deposits_matured=0</code> , <code>loans_marked_defaulted=0</code> , <code>status=completed</code>	<code>SELECT * FROM public.cob_runs WHERE cob_run_id=:id;</code>	P2

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-COB-002	Tự động đáo hạn sổ tiết kiệm	Mở sổ với <code>start_date</code> quá khứ để <code>maturity_date</code> ≤ hôm nay, rồi chạy COB	Sổ tự chuyển <code>matured</code> (không cần gọi <code>/mature</code> tay)	<code>SELECT status, closed_date FROM public.term_deposits WHERE term_deposit_id=:id; → matured</code>	P1
TC-COB-003	Biên nợ xấu — đúng 90 ngày	Khoản vay có kỳ quá hạn đúng 90 ngày	Không chuyển <code>defaulted</code> (điều kiện <code>>90</code> , không phải <code>≥90</code>)	<code>SELECT status FROM public.loans WHERE loan_id=:id; → vẫn active; SELECT CURRENT_DATE - due_date AS days_overdue FROM public.loan_payments WHERE loan_id=:id AND paid_date IS NULL; → xác nhận đúng 90</code>	P1
TC-COB-004	Biên nợ xấu — 91 ngày	Khoản vay có kỳ quá hạn 91 ngày	Chuyển <code>status=default</code>	<code>SELECT status FROM public.loans WHERE loan_id=:id; → defaulted</code>	P1
TC-COB-005	Log lịch sử chạy	<code>GET /cob/runs</code> sau vài lần chạy	200, mỗi lần chạy có <code>run_date</code> , số liệu đúng	<code>SELECT cob_run_id, run_date, term_deposits_matured, loans_marked_defaulted FROM public.cob_runs ORDER BY cob_run_id DESC;</code>	P2
TC-COB-006	Xem chi tiết 1 lần chạy	<code>GET /cob/runs/:id</code>	200, <code>details</code> JSON có danh sách sổ/vay/thẻ đã xử lý (và <code>matured_errors</code> nếu có lỗi cục bộ)	<code>SELECT details FROM public.cob_runs WHERE cob_run_id=:id;</code>	P3
TC-COB-007	COB tự động theo cron (kiểm thử tích hợp)	Đặt <code>COB_CRON_SCHEDULE=*</code> * * * *, khởi động lại server, đợi 1 phút	Log server tự in "Hoàn tất #..." không cần gọi API tay	<code>SELECT MAX(cob_run_id), MAX(started_at) FROM public.cob_runs; → xuất hiện dòng mới không do QA gọi API</code>	P3

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-COB-008	Tự động chuyển thẻ hết hạn	Phát hành thẻ (<code>expiry_date</code> = ngày mai, vì API chặn phát hành thẻ hết hạn ngay từ đầu), rồi chạy <code>POST /cob/run {as_of_date: "<ngày mới>"}</code> để giả lập đã qua ngày hết hạn	Thẻ tự chuyển <code>expired</code> , <code>expired_cards</code> trong response có ID thẻ này	<code>SELECT status FROM public.cards WHERE card_id=:id; → expired</code>	P1
TC-COB-009	Chạy COB 2 lần liên tiếp (idempotency toàn batch)	Chạy <code>/cob/run</code> khi đang có sổ/vay/thẻ thật sự đến hạn (lần 1 xử lý được), gọi <code>/cob/run</code> lần 2 ngay sau đó cùng <code>as_of_date</code>	Lần 2: <code>term_deposits_matured=0</code> , <code>loans_marked_defaulted=0</code> , <code>expired_cards=[]</code> — không xử lý trùng lại các bản ghi lần 1 đã xử lý xong	<code>SELECT COUNT(*) FROM public.cob_runs WHERE run_date=:asOf; → 2 dòng</code> (2 lần chạy đều được ghi log), nhưng <code>SELECT status FROM public.term_deposits/loans/cards WHERE ... →</code> trạng thái giữ nguyên như sau lần 1, không đổi thêm	P1

7.13 FR-UI — Giao diện quản trị (*smoke test*)

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-UI-001	Trang chủ tài được	Mở <code>http://localhost:3000/</code>	200, đủ 12 mục sidebar	(không cần)	P2
TC-UI-002	Đăng nhập Maker/Checker trong tab Duyệt lệnh	Nhập <code>teller1 / Demo@123</code> vào ô Maker	Hiện thị "Đang đăng nhập: ..." đúng tên	(không cần)	P2
TC-UI-003	Tạo khách hàng qua UI	Điền form "Tạo khách hàng mới"	Bảng chi tiết hiển thị đúng dữ liệu vừa tạo	<code>SELECT * FROM public.customers WHERE email=:email;</code>	P3
TC-UI-004	Mở sổ tiết kiệm qua UI, xem lãi dự thu	Thao tác trên tab Tiền gửi	Số liệu khớp với gọi API trực tiếp	<code>SELECT principal_amount, interest_rate FROM public.term_deposits WHERE term_deposit_id=:id;</code>	P3
TC-UI-005	Chạy COB qua nút bấm	Tab Batch cuối ngày → "Chạy COB ngay"	Lịch sử chạy cập nhật thêm 1 dòng	<code>SELECT COUNT(*) FROM public.cob_runs; → tăng thêm 1</code>	P3

7.14 FR-DOC — Tài liệu API

ID	TIÊU ĐỀ	BƯỚC / DỮ LIỆU	KẾT QUẢ MONG ĐỢI	KIỂM CHỨNG QUA DB (SQL)	ƯU TIÊN
TC-DOC-001	Swagger UI tải được	Mở <code>http://localhost:3000/docs</code>	200, hiển thị đủ 14 nhóm tag	<i>(không cần)</i>	P2
TC-DOC-002	Thử 1 endpoint qua "Try it out"	Chọn <code>GET /account-types</code> → Execute	Trả đúng response như gọi trực tiếp	<code>SELECT * FROM public.account_types;</code>	P3

8. Ma trận truy vết (Traceability Summary)

MODULE (SRS)	SỐ TEST CASE	KHOẢNG ID
4.1 FR-REF	5	TC-REF-001...005
4.2 FR-CUST	14	TC-CUST-001...014
4.3 FR-ACC	11	TC-ACC-001...011
4.4 FR-CARD	5	TC-CARD-001...005
4.5 FR-TXN	20	TC-TXN-001...020
4.6 FR-TD	21	TC-TD-001...021
4.7 FR-LOAN	12	TC-LOAN-001...012
4.8 FR-FRAUD	8	TC-FRAUD-001...008
4.9 FR-FX	11	TC-FX-001...011
4.10 FR-GL	7	TC-GL-001...007
4.11 FR-AUTH	18	TC-AUTH-001...018
4.12 FR-COB	7	TC-COB-001...007
4.13 FR-UI	5	TC-UI-001...005
4.14 FR-DOC	2	TC-DOC-001...002
Tổng	146	

9. Rủi ro & giả định khi kiểm thử

- **Test phụ thuộc thời gian** (TC-TD-007...018, TC-COB-002...004): phải tính ngày động (`hôm nay ± N ngày`) khi tạo dữ liệu test, không hard-code ngày cụ thể — nếu không bộ test sẽ pass/fail sai theo thời gian chạy thực tế.
- **Thứ tự chạy**: một số test case tạo dữ liệu dùng cho test khác (vd TC-ACC-001 tạo account cho TC-TXN-*) — nên chạy theo nhóm module tuần tự, không song song trong cùng 1 nhóm.
- **TC-GL-002 là bất biến xuyên suốt**: khuyến nghị chèn bước gọi `GET /gl/trial-balance` sau **mỗi nhóm module** (không chỉ 1 lần cuối) để khoanh vùng chính xác thao tác nào (nếu có) làm lệch sổ cái.
- **TC-COB-007** cần khởi động lại server với biến môi trường khác — nên tách chạy riêng, không nằm trong bộ smoke test thường xuyên.
- Môi trường Supabase dùng chung — tránh chạy song song 2 bộ test cùng lúc gây nhiễu dữ liệu số dư giữa các test case.